Gaussian Elimination with Partial Pivoting

Standard Gaussian Elimination Overview

Standard Gaussian Elimination is a classical direct method for solving a linear system of algebraic equations $\mathbf{Ax} = \mathbf{b}$. The core idea is to transform the system into an equivalent, simpler form without changing its solution. This is accomplished in two sequential stages:

1. **Forward Elimination:** Using elementary row operations, the augmented matrix $[\mathbf{A} \mid \mathbf{b}]$ is converted into an upper-triangular form $[\mathbf{U} \mid \mathbf{c}]$ by eliminating all coefficients located below the main diagonal.
2. **Back Substitution:** Starting from the last row (which contains only one unknown), the solution vector $\mathbf{x}$ is calculated backwards, step-by-step, up to the first variable.

What is the Pivot Element?

During Forward Elimination at stage k (working on the k-th column), the diagonal entry $a_{k,k}$ is called the **pivot element** (or simply the **pivot**), and the corresponding row k is referred to as the **pivot row**.

To eliminate an entry $a_{i,k}$ located below the pivot in row i ($i > k$), we calculate an elimination factor known as the **multiplier**:

$$m_{i,k} = \frac{a_{i,k}}{a_{k,k}}$$

We then subtract $m_{i,k}$ times the pivot row from row i to introduce a zero at position (i, k).

Why Do We Need Partial Pivoting?

In the standard approach, we directly divide by the diagonal element $a_{k,k}$. This creates two major computational problems:

1. **Division by Zero:** If $a_{k,k} = 0$, the multiplier $m_{i,k}$ cannot be computed, causing the algorithm to crash completely—even if the system has a unique, well-defined solution.
2. **Round-off Error Amplification:** If $a_{k,k}$ is not zero but is very small in magnitude compared to the coefficients below it ($|a_{k,k}| \ll |a_{i,k}|$), the multiplier becomes very large ($|m_{i,k}| \gg 1$). In finite-precision floating-point arithmetic, multiplying an entire row by a large multiplier magnifies round-off errors and leads to severe numerical inaccuracies.

The Partial Pivoting Strategy

To resolve these issues, **Partial Pivoting** (row pivoting) is applied at each elimination step k:

1. **Search for the Maximum Magnitude:** Before eliminating column k, the algorithm scans all elements in column k from the diagonal down to the bottom row to locate the coefficient with the largest absolute value:

$$p = \arg \max_{m \in \{k, k+1, \ldots, N-1\}} |a_{m,k}|$$

2. **Row Swap:** If this maximum element is located in a row p different from row k ($p \neq k$), row k and row p are completely swapped across both matrix $\mathbf{A}$ and vector $\mathbf{b}$.

By placing the largest available entry on the diagonal, every multiplier is guaranteed to satisfy:

$$|m_{i,k}| = \left| \frac{a_{i,k}}{a_{k,k}} \right| \leq 1, \quad \forall i > k$$

This bounds the multipliers, prevents division by zero, and ensures numerical stability throughout the computation.

C++ Implementation of Gaussian Elimination with Partial Pivoting

This section describes the object-oriented implementation of the Gaussian Elimination algorithm with Partial Pivoting in the `GaussianElimination` class.

The implementation follows the **Separation of Concerns** principle. Each important step of the algorithm is encapsulated in a separate method, making the code easier to understand, test, maintain, and extend.

`FindPivotRowIndex` Method

Conceptual Role Of `FindPivotRowIndex` The objective of this method is to implement the row pivot selection strategy during elimination step k. To minimize round-off errors and prevent potential division by zero, the method traverses column k from the main diagonal entry (k, k) down to the final row $(N-1, k)$, identifying the row that contains the maximum absolute coefficient:

$$\text{PivotRow} = \arg \max_{i \in \{k, k+1, \ldots, N-1\}} |A_{i,k}|$$

C++ Implementation Of `FindPivotRowIndex`

```
sstd::size_t GaussianElimination::FindPivotRowIndex(
    const CoefficientMatrix& A,
    const std::size_t PivotIndex)
{
    const std::size_t NumRows = A.GetNumRows();

    // Ensure the pivot index is within matrix bounds
    if (PivotIndex >= NumRows)
    {
        throw std::out_of_range("FindPivotRowIndex: PivotIndex is out of range.");
    }

    // Initialize the best candidate with the current diagonal element
```

```
    std::size_t CandidateRowIndex = PivotIndex;
    double MaxCandidateAbsValue = std::fabs(A.GetValue(PivotIndex, PivotIndex));

    // Search subsequent rows for the largest absolute pivot value
    for (std::size_t RowIndex = PivotIndex + 1; RowIndex < NumRows; ++RowIndex)
    {
        const double CurrentAbsValue = std::fabs(A.GetValue(RowIndex, PivotIndex));

        if (CurrentAbsValue > MaxCandidateAbsValue)
        {
            MaxCandidateAbsValue = CurrentAbsValue;
            CandidateRowIndex = RowIndex;
        }
    }

    return CandidateRowIndex;
}
```

`FindPivotRow` Method Summary The `FindPivotRowIndex` method searches the active column for the largest absolute coefficient, starting from the current pivot row. It returns the index of the selected row for use in partial pivoting without modifying the matrix.

`SwapRows` Method

Conceptual Role Of `SwapRows` The `SwapRows` method exchanges two rows of the coefficient matrix. During Gaussian Elimination with Partial Pivoting, it is used to move the selected pivot row into the current pivot position.

This operation ensures that the row containing the largest suitable pivot coefficient is placed at the correct position before the elimination process continues.

C++ Implementation Of `SwapRows` Method

```
void GaussianElimination::SwapRows(
    CoefficientMatrix& A,
    RHS& RHS,
    std::size_t FirstRowIndex,
    std::size_t SecondRowIndex)
{
    const std::size_t NumCols = A.GetNumColumns();

    if (FirstRowIndex >= A.GetNumRows() || SecondRowIndex >= A.GetNumRows())
    {
        throw std::out_of_range("SwapRows: row index is out of range.");
    }
```

```
    if (RHS.GetSize() != A.GetNumRows())
    {
        throw std::invalid_argument("SwapRows: size of RHS must match matrix size.");
    }

    if (FirstRowIndex == SecondRowIndex)
    {
        return;
    }

    for (std::size_t ColIndex = 0; ColIndex < NumCols; ++ColIndex)
    {
        const double Temp = A.GetValue(FirstRowIndex, ColIndex);
        A.SetValue(FirstRowIndex, ColIndex, A.GetValue(SecondRowIndex, ColIndex));
        A.SetValue(SecondRowIndex, ColIndex, Temp);
    }

    const double TempRHS = RHS.GetValue(FirstRowIndex);
    RHS.SetValue(FirstRowIndex, RHS.GetValue(SecondRowIndex));
    RHS.SetValue(SecondRowIndex, TempRHS);
}
```

`SwapRows` Method Summary The `SwapRows` method does not perform any complex calculation. It simply exchanges the coefficients of two rows, which is equivalent to swapping the positions of two equations in the linear system. In this method, FirstRowIndex represents the current pivot row in the elimination process, while SecondRowIndex corresponds to the row identified by `FindPivotRowIndex` that contains the largest absolute coefficient in the active column. The method simply swaps these two rows so that the optimal pivot is placed on the main diagonal.

`ForwardElimination` Method

Conceptual Role Of `ForwardElimination` The objective of the `ForwardElimination` method is to transform the augmented matrix $[A|b]$ into an equivalent upper triangular matrix $[U|c]$ through systematic row operations.

During each step k, the method incorporates **Partial Pivoting** by delegating pivot row selection to `FindPivotRowIndex` and row interchange to `SwapRows`. Once the optimal pivot element $A_{k,k}$ is placed on the main diagonal, multipliers (factors) are computed to eliminate all sub-diagonal entries in the active column k:

$$m_{i,k} = \frac{A_{i,k}}{A_{k,k}}, \quad \text{for } i = k+1, \dots, N-1$$

The row reduction updates both the coefficient matrix and the right-hand side vector:

$$A_{i,j} \leftarrow A_{i,j} - m_{i,k} A_{k,j}, \quad \text{for } j = k+1, \ldots, N-1$$

$$b_i \leftarrow b_i - m_{i,k} b_k$$

C++ Implementation Of `ForwardElimination` Method

```
void GaussianElimination::ForwardElimination(
    CoefficientMatrix& A,
    RHS& RHS)
{
    const std::size_t NumRows = A.GetNumRows();
    const std::size_t NumCols = A.GetNumColumns();

    if (NumRows != NumCols)
    {
        throw std::invalid_argument("ForwardElimination: matrix A must be square.");
    }

    if (RHS.GetSize() != NumRows)
    {
        throw std::invalid_argument("ForwardElimination: size of RHS must match matrix s
    }

    if (NumRows == 0)
    {
        return;
    }

    for (std::size_t PivotColIndex = 0; PivotColIndex < NumCols - 1; ++PivotColIndex)
    {
        std::size_t PivotRowIndex = PivotColIndex;

        const std::size_t Candidate_PivotRowIndex = FindPivotRowIndex(A, PivotColIndex);

        if (Candidate_PivotRowIndex != PivotRowIndex)
        {
            SwapRows(A, RHS, PivotRowIndex, Candidate_PivotRowIndex);
        }

        const double PivotValue = A.GetValue(PivotRowIndex, PivotColIndex);

        if (std::fabs(PivotValue) < 1.0e-14)
        {
```

```
            throw std::runtime_error("ForwardElimination: singular or nearly singular ma
        }


        for (std::size_t RowIndex = PivotRowIndex + 1; RowIndex < NumRows; ++RowIndex)
        {
            const double Factor = A.GetValue(RowIndex, PivotColIndex) / PivotValue;

            A.SetValue(RowIndex, PivotColIndex, 0.0);

            for (std::size_t ColIndex = PivotColIndex + 1; ColIndex < NumCols; ++ColInde
            {
                const double NewValue =
                    A.GetValue(RowIndex, ColIndex) - Factor * A.GetValue(PivotRowIndex,
                A.SetValue(RowIndex, ColIndex, NewValue);
            }

            const double NewRHS =
                RHS.GetValue(RowIndex) - Factor * RHS.GetValue(PivotRowIndex);
            RHS.SetValue(RowIndex, NewRHS);
        }
    }

    if (std::fabs(A.GetValue(NumRows - 1, NumRows - 1)) < 1.0e-14)
    {
        throw std::runtime_error("ForwardElimination: zero pivot found on last row.");
    }
}
```

`ForwardElimination` Method Summary The ForwardElimination method transforms the system into upper-triangular form using partial pivoting. It systematically computes row elimination multipliers, zeroes out sub-diagonal coefficients, and simultaneously updates the right-hand side vector RHS while detecting singular matrices.

`ForwardElimination` — Transforming the System The `ForwardElimination` method transforms the general dense linear system of Equation(**??**) into an equivalent upper triangular form shown in Equation (2).

Upper Triangular System Following the completion of the forward elimination sweeps, all entries strictly below the main diagonal vanish ($a'_{i,j} = 0$ for $i > j$):

$$\begin{bmatrix} a'_{0,0} & a'_{0,1} & a'_{0,2} & \cdots & a'_{0,N-1} \\ 0 & a'_{1,1} & a'_{1,2} & \cdots & a'_{1,N-1} \\ 0 & 0 & a'_{2,2} & \cdots & a'_{2,N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & a'_{N-1,N-1} \end{bmatrix} \begin{bmatrix} x_0^{n+1} \\ x_1^{n+1} \\ x_2^{n+1} \\ \vdots \\ x_{N-1}^{n+1} \end{bmatrix} = \begin{bmatrix} b_0'^n \\ b_1'^n \\ b_2'^n \\ \vdots \\ b_{N-1}'^n \end{bmatrix} \tag{1}$$

`BackSubstitution` Method

Conceptual Role Of `BackSubstitution` Once the forward elimination process transforms the original coefficient matrix $\mathbf{A}$ into an upper triangular matrix $\mathbf{U}$ and updates the right-hand side vector $\mathbf{b}$ accordingly, the linear system takes the form:

$$\mathbf{U}\mathbf{x} = \mathbf{b} \tag{2}$$

Expanded into individual scalar equations, the i-th row equation is written as:

$$U_{i,i}\, x_i + \sum_{j=i+1}^{N-1} U_{i,j}\, x_j = b_i \tag{3}$$

The back-substitution procedure resolves the unknowns in reverse topological order, starting from the last unknown x_{N-1} and progressing backwards to x_0. Isolating x_i yields the explicit mathematical recurrence relation:

$$x_i = \frac{b_i - \sum_{j=i+1}^{N-1} U_{i,j}\, x_j}{U_{i,i}}, \quad \text{for } i = N-1, N-2, \ldots, 0 \tag{4}$$

Where:

- i: The current row and equation index, traversed backwards from $N-1$ down to 0.
- j: Column indices strictly to the right of the main diagonal ($j > i$).
- $U_{i,i}$: The non-zero pivot entry on the main diagonal of the upper triangular system.
- c_i: The corresponding updated load value in the right-hand side vector.
- $\sum_{j=i+1}^{N-1} U_{i,j}\, x_j$: The cumulative contribution of previously computed unknowns in subsequent rows.
- x_i: The newly determined scalar solution for the i-th degree of freedom.

C++ Implementation Of `BackSubstitution` The implementation employs a signed index type (`std::ptrdiff_t`) to eliminate the risk of unsigned underflow during reverse iteration. Furthermore, singular or near-singular systems are trapped by verifying that $|U_{i,i}| \geq 1.0 \times 10^{-14}$ before performing the floating-point division.

```
// Mathematical formulation of back-substitution:
// x_i = ( b_i - \sum_{j=i+1}^{N-1} A_{i,j} * x_j ) / A_{i,i}
```

```
//
// Where:
//   i         : Current row and equation index (from N-1 down to 0)
//   j         : Column indices to the right of the main diagonal (j > i)
//   A_{i,i}   : Pivot/diagonal entry of the current row
//   b_i       : Corresponding element in the right-hand side (RHS) vector
//   sum_upper : Cumulative sum of known upper terms: \sum_{j=i+1}^{N-1} A_{i,j} * x_j
//   x_i       : Computed unknown value stored in the solution vector

void GaussianElimination::BackSubstitution(
    const CoefficientMatrix& A,
    const RHS& RHS,
    Field1D& Solution)
{
    const std::size_t NumRows = A.GetNumRows();
    const std::size_t NumCols = A.GetNumColumns();

    if (NumRows != NumCols)
    {
        throw std::invalid_argument("BackSubstitution: matrix A must be square.");
    }

    if (RHS.GetSize() != NumRows || Solution.Size() != NumRows)
    {
        throw std::invalid_argument("BackSubstitution: size of RHS and Solution must mat
    }

    // Mathematical formula:
    // x_i = ( b_i - \sum_{j=i+1}^{N-1} A_{i,j} * x_j ) / A_{i,i}

    // Iterate backward from the last row (N-1) down to the first row (0)
    for (std::ptrdiff_t row = static_cast<std::ptrdiff_t>(NumRows) - 1; row >= 0; --row)
    {
        const std::size_t i = static_cast<std::size_t>(row);

        // Calculate the sum of known upper terms: \sum_{j=i+1}^{N-1} A_{i,j} * x_j
        double sum_upper = 0.0;
        for (std::size_t j = i + 1; j < NumCols; ++j)
        {
            sum_upper += A.GetValue(i, j) * Solution.GetValue(j);
        }

        const double A_ii = A.GetValue(i, i);

        if (std::fabs(A_ii) < 1.0e-14)
```

```
        {
            throw std::runtime_error("BackSubstitution: zero diagonal entry detected.");
        }

        const double b_i = RHS.GetValue(i);

        // x_i = (b_i - sum_upper) / A_ii
        Solution.SetValue(i, (b_i - sum_upper) / A_ii);
    }
}
```